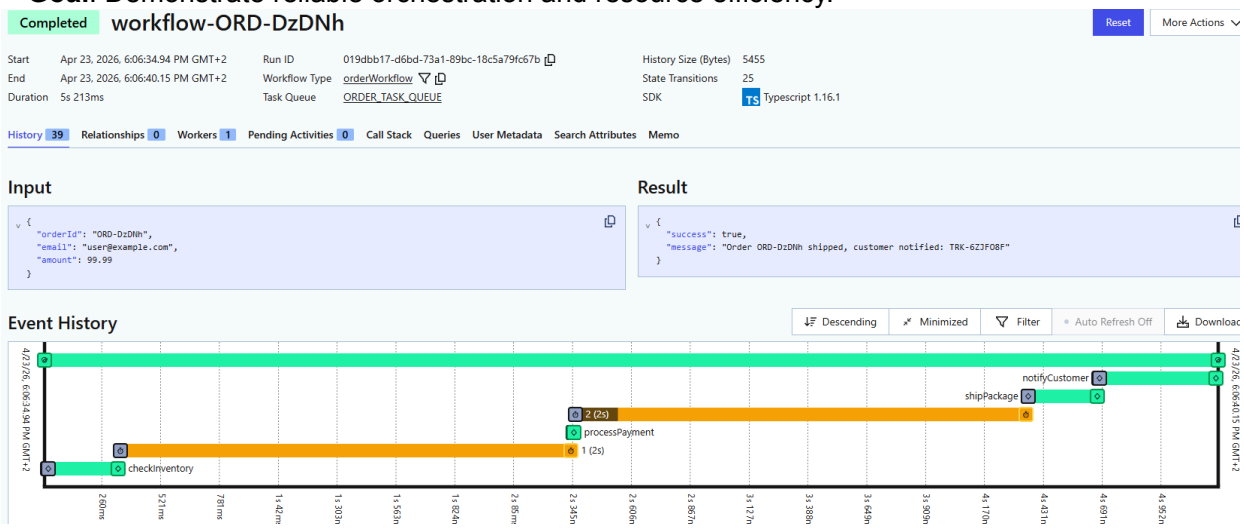
Temporal E-Commerce Order Service

An implementation of the **Saga Pattern** for distributed transactions using the Temporal TypeScript SDK. Complete steps are included in the README.md in code repository

Scenario 1: The Green Path

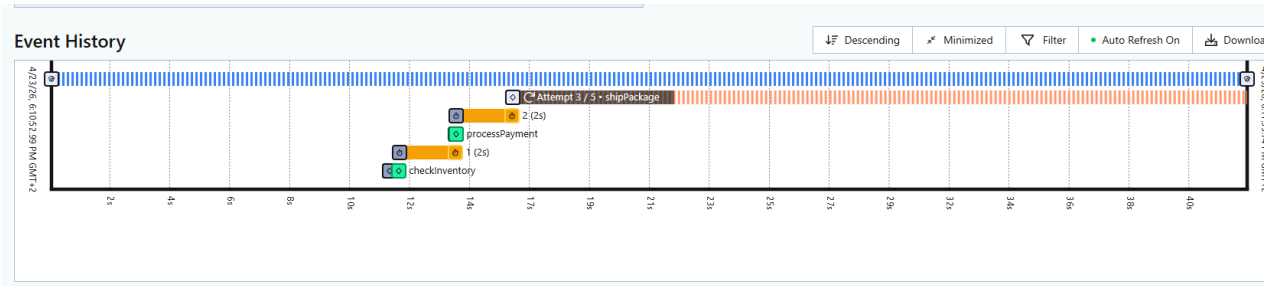
- **Goal**: Demonstrate reliable orchestration and resource efficiency.



- **Info**: A 2-second sleep has been added between activities, for demo purposes and switching screens delay. The state is persisted, so if the worker goes idle, the History Service will wake it back up

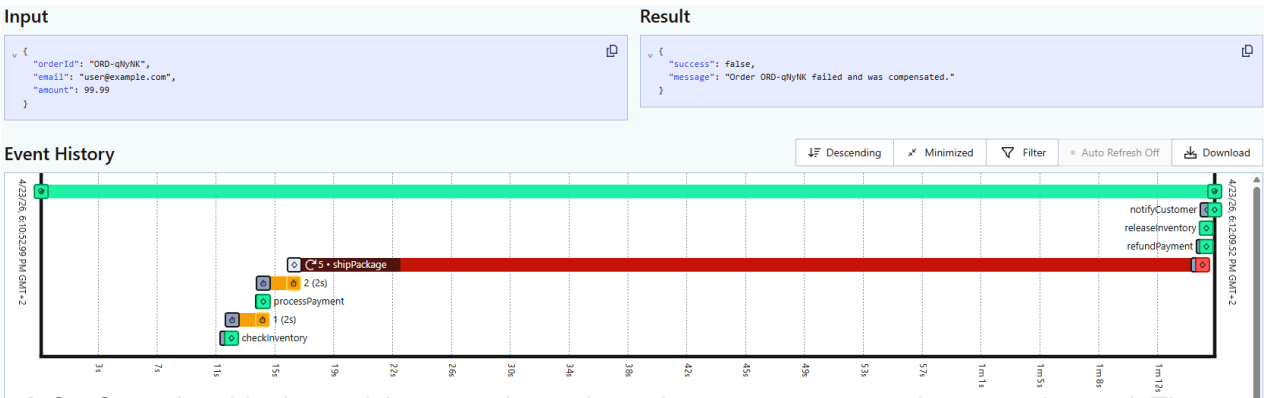
Scenario 2: The Saga Pattern (Handling Terminal Failures)

- **Goal**: Demonstrate eventual consistency and automated cleanup.



All | Compact | JSON

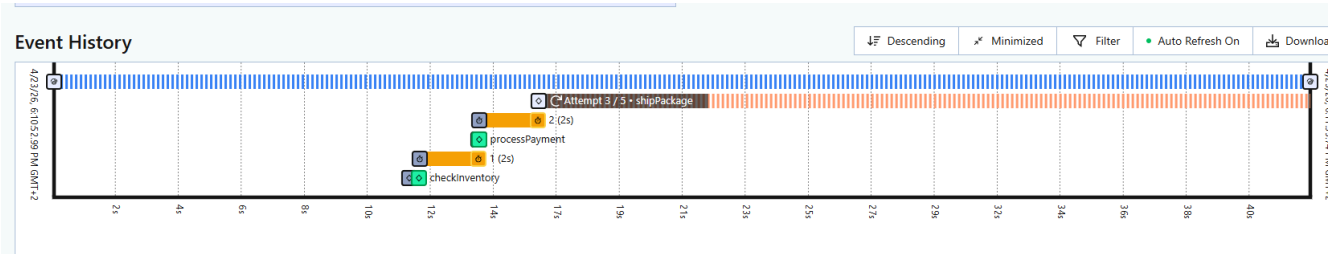
Event ID	Timestamp	Event Type	Details
27	Apr 23, 2026, 6:11:09.47 PM GMT+2	Pending Activity	Attempt 3 / 5 • Next Retry 3s Activity Type shipPackage
27	Apr 23, 2026, 6:11:09.47 PM GMT+2	Activity Task Scheduled	Activity Type shipPackage
26	Apr 23, 2026, 6:11:09.47 PM GMT+2	Workflow Task Completed	Identity 39728@Legion5

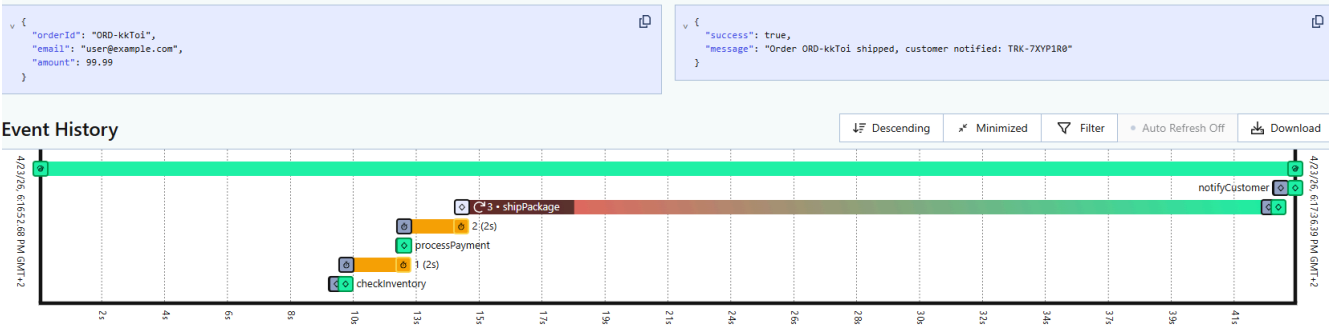


- **Info**: Once the shipping activity exceeds maximumAttempts, compensations are triggered. The payment is refunded and releases inventory in reverse order to ensure the system doesn't stay in an inconsistent state."

Scenario 3: The "Hot-Fix" (Durable Recovery)

- **Goal**: Demonstrate recovery of in-flight business logic without data loss.





- **Info**: As the workflow state lives in the **Persistence Store** (SQLite) and not in the worker's memory, new code can be deployed and restart my infrastructure without the user's order ever failing or needing a manual restart.